UE24CS242B: Operating Systems

Operating Systems Banana Problem

> We encourage you to look things up and explore as you go when needed. Using ChatGPT to help understand concepts and learn is perfectly fine, but please avoid using it to generate or print out final answers.

The Hive Containment Protocol

Prerequisites:

This lab requires a POSIX-compliant environment (Linux or macOS).
Virtual Machine: Ubuntu LTS versions are recommended.
Compiler: Ensure `gcc` is installed.

Quick check:

```
gcc --version
```

Introduction & Conceptual Context

Before we dive into the terminal, let's understand the core Operating System concepts you will be interacting with today.

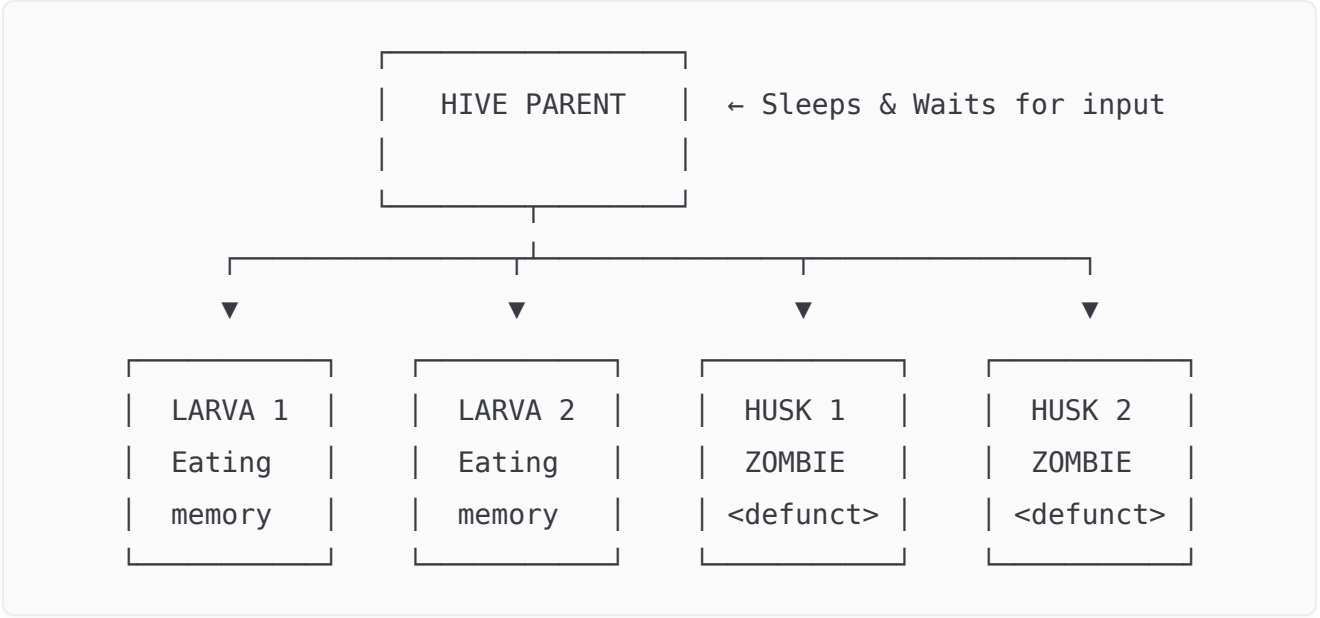
In Linux, processes are not solitary entities; they exist in a hierarchy. A **Parent** process spawns **Child** processes using system calls like `fork()`. Usually, a parent is responsible for "reaping" its children-acknowledging that they have finished their work so the system can clean up their resources.

However, things can go wrong:

1. **Zombies:** If a child process finishes execution (exits) but the parent is busy or poorly programmed and fails to acknowledge it (via `wait()` or `waitpid()`), that child becomes a **Zombie**. It is dead code, but it still occupies a slot in the system's process table.
2. **Orphans:** If a parent dies before the child, the child becomes an orphan (usually

adopted by the `init` process).

In this lab, you will act as a system administrator debugging a rogue program called **"The Hive"**. This program demonstrates erratic behavior: it spawns memory-hungry children and leaves behind zombie processes. Your goal is to analyze its behavior, manage its resources, and terminate it gracefully using standard Linux commands.



What's Happening?:

- **What the Hive does**: The Hive is the **parent** process. It creates two kinds of children:
  - **Larvae**: long-running children that keep allocating memory (they often show up as **sleeping** because they spend time in `sleep(2)` between allocations).
  - **Husks**: children that **exit immediately** and become **zombies** until the parent reaps them.
- **Two terminals**: one terminal runs the Hive and accepts your commands; the other is your “scanner” for observing the process tree.

The Scenario

A rogue process-codenamed **"The Hive"**-has been detected on the server. It behaves like a biological organism:

1. It spawns **Larvae**: Active child processes that continuously eat up system memory.
2. It leaves behind **Husks**: Child processes that die immediately but are never cleaned up, turning into **Zombies**.

Your Mission:

1. **Analyze** the provided `hive.c` source code to understand its internal logic.
2. **Hunt** the Hive using terminal commands. Identify its PID, track its resource usage, and find its children.
3. **Contain** the threat by issuing specific commands to trigger cleanup routines and eventually terminate the process.

> Please submit a single **PDF** containing both the screenshots and your answers to the questions below. Clearly label each section of the PDF with the appropriate heading, and include the screenshot number alongside each screenshot. This will help make the submission easy for us to review and assess <3
> In the end, please leave a short feedback in the google form link as to what you thought about the assignment. Healthy criticism is much welcome.

The Source Code: `hive.c`

Instructions: Given a file named `hive.c` explore it while doing the assignment. **Do not modify this code.**

> It's a good practice to start reading the code from the main function.

Part A: The Hunt

For this section, you will use two terminal windows.

- **Terminal 1:** Runs the Hive (and is also your control console where you control and hunt THE HIVE).
- **Terminal 2:** The "Hunter" terminal where you run your analysis commands.

Document your progress by taking screenshots when specified.

Phase 1: Deployment & Forensics

1. **Compile and Run** In Terminal 1, compile `hive.c` using `gcc` and run it with your SRN as an argument. *(Screenshot 1: The Hive startup message)*

- **What to look for**: the Hive prints its **PID** and the values of `L` and `Z` (how many Larvae and Husks it will create).

2. **Locate the Target** The Hive writes its Process ID (PID) to a file on the disk. Analyze the C code to find the path where this file is created. Use a common linux shell command used to print out the contents of
that file on the terminal. Read this file and obtain the PID. *(Screenshot 2: The PID file contents)*
3. **Map the Family Tree** In Terminal 2, use a process listing command to find all children processes belonging to the Hive. *(Screenshot 3: Process listing showing all hive children)*
   - **What to look for**: multiple child processes, with `PPID` equal to the Hive's PID.
   - Look for children in state `S` (Sleeping/Interruptible wait) - these are Larvae.
   - Look for children in state `Z` (Zombie/Defunct) - these are Husks.
4. **Isolate the Undead** Filter your process list to show **only** the Zombie (`Z`) processes. Are there multiple ways to do it? *(Screenshot 4: List of zombie processes)*
   - **Note**: zombies are the undead - they don't run, but they take up a process table entry until reaped.
5. **Analyze Memory Consumption** Identify the Larvae. Display their memory usage (specifically **RSS** - Resident Set Size, i.e., the memory currently resident in RAM). *(Screenshot 5: Memory usage for all larvae)*
   - **What to look for**: Larvae should show non-trivial RSS and typically grow over time; after allocation failure they go dormant (`pause()`), after which RSS often stabilizes.
6. **Investigate Logs** The code creates log files for each Larva in a specific directory. Locate this directory from the code (look for where log file paths are constructed). Open one of the log files to see the memory allocation history. *(Screenshot 6: Contents of one larva log file)*
   - **What to look for**: repeated "Allocated 1 MB" lines over time.
7. **Verify Parent/Child Links (PID/PPID)** You know the Hive PID. Show the **PPID** of each child and verify that the Hive is their parent. (Hint: look for `PID` and `PPID` columns, or use a tree view.)

Phase 2: The Cure

The Hive has a weak spot for specific commands.

1. **Reap the Zombies** In Terminal 1 (the Hive console), type `reap` and press Enter. Observe the message printed by the Hive. *(Screenshot 7: Terminal 1 showing the*

"Command received: reap" message)

- **What you are doing**: you are making the **parent** clean up its exited children.

2. **Verify Cleanup** Run your command to list Zombie processes again. The list should now be empty. *(Screenshot 8: Output showing zero zombies)*
3. **Check the Larvae** Verify that the Larvae processes are still alive and running. The `reap` command should have only affected zombies. *(Screenshot 9: Remaining children listing)*
    - **Sanity check**: Larvae should still appear as `S` (sleeping), not `Z`.

Phase 3: Sterilization

1. **Terminate the Hive** In Terminal 1 (the Hive console), type `quit` and press Enter. Observe Terminal 1. The parent should clean up remaining children and exit. *(Screenshot 10: Terminal 1 showing shutdown message)*
    - **What good looks like**: the Hive (parent) exits, and the larvae should exit shortly after.

Part B: Code Analysis (Theory)

Read the hive code carefully to answer the following questions. Write the answers to these in the submission PDF.

1. Verifying Parent/Child Relationships (PID/PPID)

Question:
How can you verify the **parent/child relationship** between the Hive and its children? Explain what **PID** and **PPID** mean, and describe one command/output that shows that each larva/husk belongs to the Hive.

Think about:

- What does `PPID` tell you?
- How can a process tree view help?

---

2. Parent Responsibility & Zombie Reaping

Question:
Why do zombies exist until the parent reaps them? Explain why `wait()` / `waitpid()`

belongs in the **parent**, what happens if the parent never reaps, and what `WNOHANG` changes compared to a blocking wait.

Think about:

- What is the difference between a zombie and a running process?
- Why might a parent want a non-blocking wait?

---

3. Zombie Creation

Question:
Explain the lifecycle of the “Husk” processes. Why do they become zombies? Identify the parent-side code (or missing code) responsible for this.

Think about:

- What does the husk child do immediately after `fork()`?
- When does the parent call `wait()` or `waitpid()`?
- What condition triggers zombie reaping?

---

4. Dormant Larvae

Question:
Why does a Larva enter `while (1) pause();` after `malloc()` fails instead of calling `exit()`? What would happen if all larvae exited while the parent was sleeping? Would this defeat the evil plan of the Hive?

Think about:

- Difference between a paused process and an exited one.
- Visibility in the process table (`ps`).

Submission Guidelines

Create a pdf named `<Your_SRN>_<Section>.pdf` (Eg: `PESxUGxxCSxxx_X.pdf`) containing:

1. **Paste** screenshots 1 through 10 directly into the document, ensuring they are legible and labeled with your command line terminal clearly visible.
2. Answers to Part B
3. Leave a short review, let us know what you felt about the assignment. Healthy criticism is much welcome

> FORM: https://forms.gle/qm2Xb3RCk2ThqEp18
